

Programa12.Regresion.OLS

April 7, 2026

Nombre: Gabriel Salguero **Fecha:** 06/04/2026

```
[15]: import numpy as np
      from math import sqrt
      from pprint import pprint
      from sklearn import datasets, linear_model, metrics
      from sklearn.dummy import DummyRegressor
      from sklearn.model_selection import cross_validate, KFold, cross_val_predict,
      ↪ train_test_split, cross_val_score
      from sklearn import preprocessing
      from sklearn.metrics import make_scorer, mean_squared_error
      import matplotlib.pyplot as plt
```

```
[16]: # Carga de datos.
      #datos = datasets.load_boston()
      datos = datasets.fetch_california_housing()
      #print(datos)
      X = datos.data
      y = datos.target
      print('Dimensiones de X: ', np.shape(X))
      # print(y)
```

Dimensiones de X: (20640, 8)

```
[17]: # # Métricas de evaluación.
      metricas = {
          'MAE': 'neg_mean_absolute_error',
          'RMSE': make_scorer(lambda y, y_pred:
                              sqrt(metrics.mean_squared_error(y, y_pred)),
                              greater_is_better=False),
          'MAPE': make_scorer(lambda y, y_pred:
                              np.mean(np.abs((y - y_pred) / y)) * 100,
                              greater_is_better=False),
          'R2': 'r2',}
```

```
[18]: # 1) Partición de datos externa
      X_training, X_testing, y_training, y_testing = train_test_split(X, y,
      ↪ test_size=0.2, random_state=42)
```

```
print([np.shape(X_training), np.shape(X_testing)])
```

```
[(16512, 8), (4128, 8)]
```

```
[21]: # 2) Extracción de características
# 3) Selección de atributos
# 4) Estandarización de los datos de entrenamiento
standardizer = preprocessing.StandardScaler()
stdr_trained = standardizer.fit(X_training)
X_stdr = stdr_trained.transform(X_training)
```

```
[22]: # 5) Construcción del algoritmo de aprendizaje.
reg = linear_model.LinearRegression(fit_intercept=True)
```

```
[23]: # 5.1) Validación cruzada interna

# Extraer directamente resultados del error para cada bolsa, en lugar de las
# predicciones
cross_val_results = cross_validate(reg, X_stdr, y_training,
                                   cv = KFold(n_splits=5, shuffle=True,
                                   random_state=42), scoring=metrics)
#print("cross_val_MAE:  %0.4f +/- %0.4f" % (-cross_val_results.mean(),
#      cross_val_results.std()))
pprint(cross_val_results)

{'fit_time': array([0.00644708, 0.00439167, 0.00427842, 0.00362778,
0.00348973]),
 'score_time': array([0.00235009, 0.00171876, 0.00210238, 0.00178075,
0.00153136]),
 'test_MAE': array([-0.54071407, -0.52878981, -0.51274193, -0.53512422,
-0.52793364]),
 'test_MAPE': array([-31.74769388, -31.65516613, -30.97054077, -32.61432353,
-30.685065  ]),
 'test_R2': array([0.60970239, 0.60411343, 0.6354319 , 0.60076148, 0.60727452]),
 'test_RMSE': array([-0.73389779, -0.72516701, -0.69726867, -0.73337185,
-0.71284225])}
```

```
[24]: # 5.2) Entrenamiento con todos los datos para obtener los coeficientes del
# modelo.
model = reg.fit(X_stdr, y_training)

# Obtención de los coeficientes del modelo.
w = model.coef_
print('Model coefficients: \n', w)

# Obtención del término independiente del modelo.
w_0 = model.intercept_
print('\nTérmino independiente: ', w_0)
```

Model coefficients:

```
[ 0.85438303  0.12254624 -0.29441013  0.33925949 -0.00230772 -0.0408291
-0.89692888 -0.86984178]
```

Término independiente: 2.071946937378619

```
[25]: # ---- PREDICCIÓN ---- #
```

```
[26]: # 6) Extracción de las características de test
# 7) Selección de los atributos de test
# 8) Estandarización de las característiacs de test
X_test_std = stdr_trained.transform(X_testing)
```

```
[27]: # 9) Predicción del conjunto de test
y_pred_test = model.predict(X_test_std)
```

```
[28]: # Cálculo de las métricas de evaluación.
MAE = metrics.mean_absolute_error(y_testing, y_pred_test)
MSE = metrics.mean_squared_error(y_testing, y_pred_test)
RMSE = metrics.mean_squared_error(y_testing, y_pred_test)
MAPE = metrics.mean_absolute_percentage_error(y_testing, y_pred_test)
R2 = metrics.r2_score(y_testing, y_pred_test)

print('MAE: %.4f' % MAE)
print('MSE: %.4f' % MSE)
print('RMSE: %.4f' % RMSE)
print('MAPE: %.4f' % MAPE)
print('R2: %.4f' % R2)
```

MAE: 0.5332

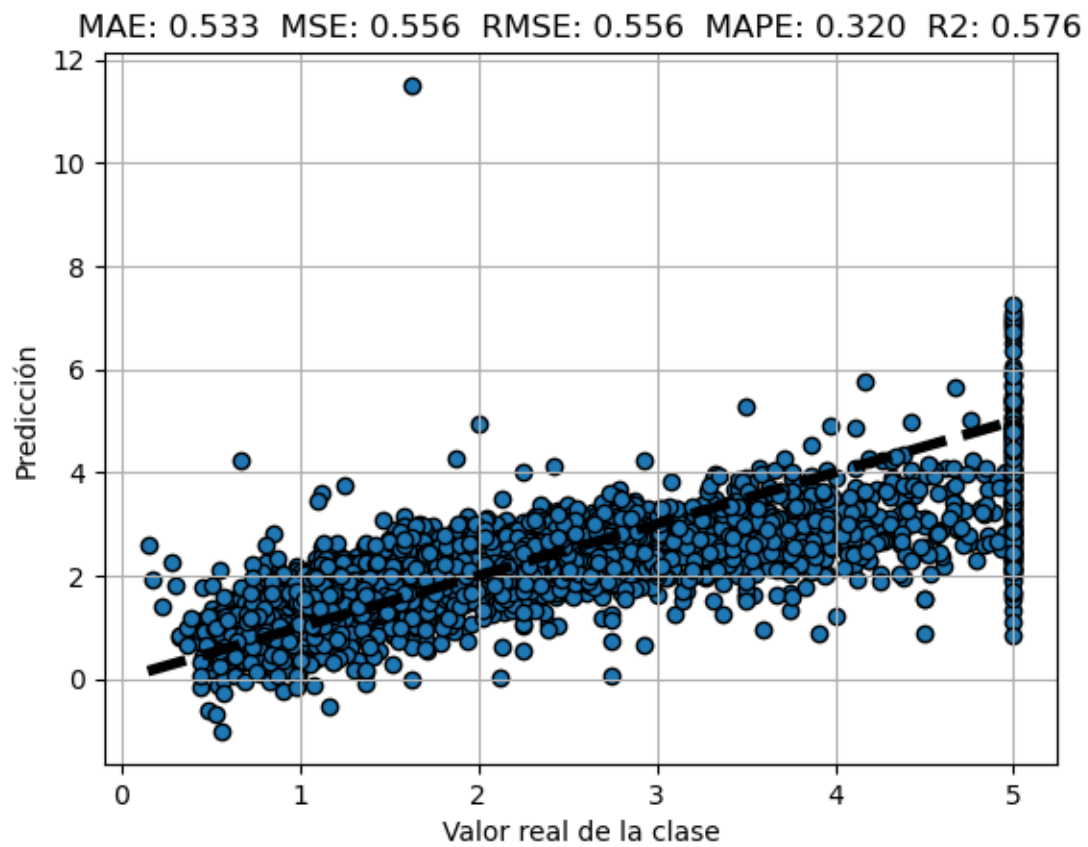
MSE: 0.5559

RMSE: 0.5559

MAPE: 0.3195

R2: 0.5758

```
[29]: # Gráfica de realidad vs. predicción.
fig, ax = plt.subplots()
ax.scatter(y_testing, y_pred_test, edgecolors=(0, 0, 0))
ax.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=4)
ax.set_xlabel('Valor real de la clase')
ax.set_ylabel('Predicción')
plt.title("MAE: %.3f MSE: %.3f RMSE: %.3f MAPE: %.3f R2: %.3f" %
          (MAE, MSE, RMSE, MAPE, R2))
plt.grid()
plt.show()
```



<https://github.com/reromash1/machines2>

[]: